An Empirical Study of Serverless Performance

Prince Keshri
B.Tech CSE (2029)
National Institute of Technology, Durgapur
https://www.linkedin.com/in/princekeshri00/

March 18, 2026

https://github.com/princekeshri00/serverless-performance-research

Abstract

Serverless computing enables developers to deploy applications without managing infrastructure, but introduces performance variability due to dynamic resource allocation. This paper presents a comprehensive empirical study of serverless performance using Google Cloud Run. The study investigates cold start behavior, image size impact, concurrency scaling, autoscaling saturation, and regional latency. Unlike conventional evaluations that focus only on average latency, this work emphasizes variability, scaling delays, and non-linear system behavior observed under realistic workloads.

1 Introduction

Serverless computing abstracts infrastructure management while enabling automatic scaling. However, this abstraction introduces variability in performance due to factors such as cold starts, scaling delays, and distributed infrastructure behavior.

In real-world applications, consistency is often more important than average performance. Tail latency and unpredictable spikes can significantly degrade user experience. Therefore, understanding the causes behind performance variability is essential.

This study aims to analyze serverless performance beyond surface-level metrics, focusing on the underlying mechanisms that drive observed behavior.

2 Background and Related Work

Previous research has identified cold start latency as a key issue in serverless platforms such as AWS Lambda and Google Cloud Run. Techniques such as pre-warming and container reuse have been proposed to mitigate these effects.

However, most studies focus on individual variables in isolation. In contrast, this work evaluates multiple interacting factors including concurrency, autoscaling, and geographic distribution.

3 Methodology

Experiments were conducted using Google Cloud Run with a Python-based containerized service. A benchmarking client generated controlled workloads under varying conditions.

Metrics collected include:

- Latency (milliseconds)
- Throughput (requests per second)
- Error rate

Each experiment isolates a specific variable to analyze its impact on performance.

4 Experiments

The following experiments were conducted:

- Cold Start vs Warm Latency
- Image Size vs Startup Latency
- Concurrency Scaling
- Autoscaling Saturation
- Regional Latency

5 Results and Analysis

5.1 Cold Start vs Warm Latency

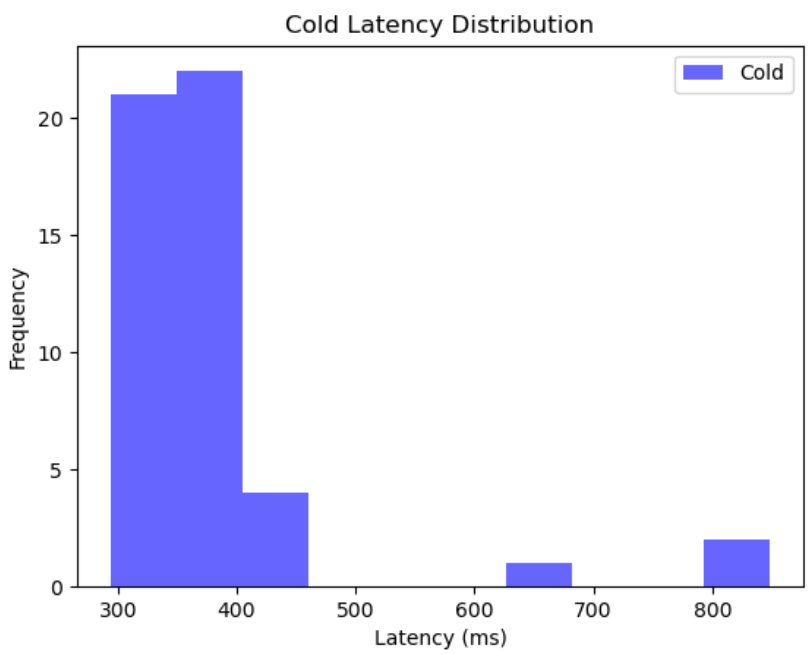
Cold start latency is one of the most critical performance concerns in serverless computing. It occurs when an incoming request is assigned to a new container instance that must be initialized from scratch. This process involves multiple stages, including resource allocation, container startup, dependency loading, and runtime initialization. Each of these stages introduces variability, making cold start behavior inherently inconsistent.

Table 1: Cold vs Warm Latency

Type	Mean (ms)	Std Dev (ms)	Max (ms)
Cold	383.9	111.1	848.8
Warm	333.0	51.0	557.1

The table highlights that the average latency difference between cold and warm requests is present but not extremely large. However, a more important observation lies in the standard deviation and maximum latency values. Cold requests exhibit significantly higher variance and extreme outliers, indicating inconsistent performance.

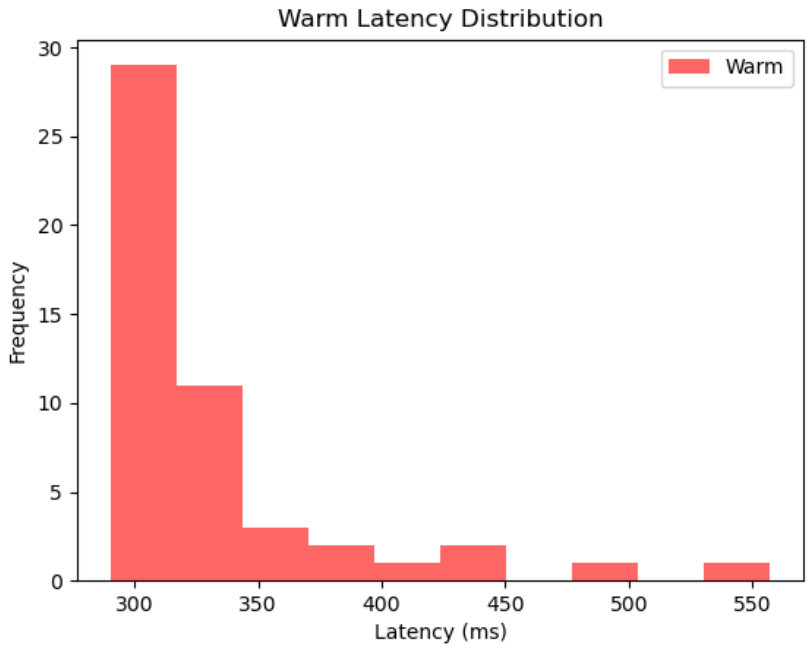
This behavior is better understood through the latency distribution of cold requests.



The distribution shows that most cold requests fall within a relatively narrow latency band, but a small number of requests experience significantly higher delays. These outliers

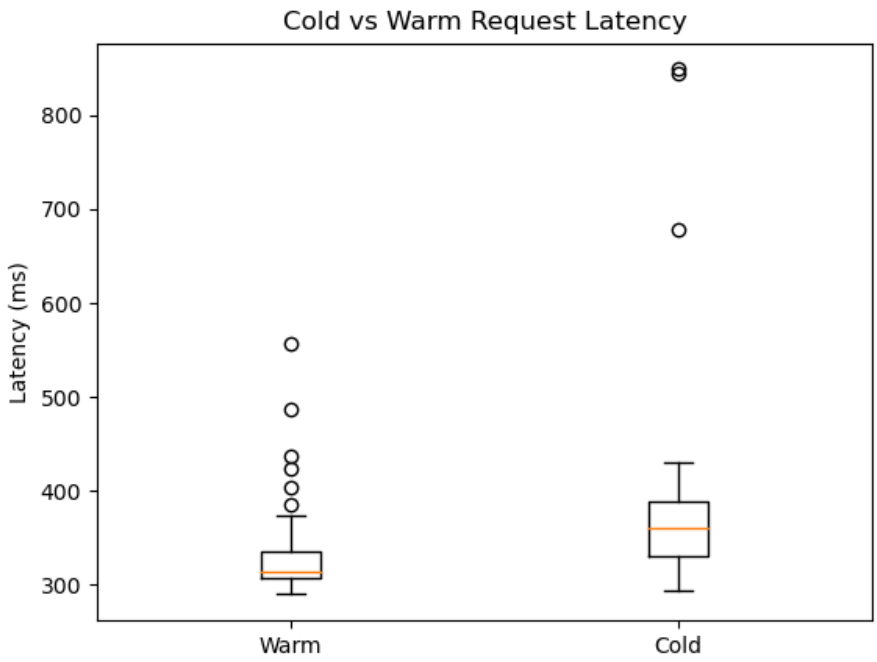
are caused by variability in the initialization process, such as delays in container provisioning, image retrieval, and scheduling within the cloud infrastructure.

Warm requests, on the other hand, are handled by already running instances, eliminating the need for initialization.



The warm latency distribution is much more tightly clustered, with fewer extreme values. This indicates that once an instance is active, the system provides stable and predictable performance.

A direct comparison between cold and warm latency further illustrates the difference in variability.



While the median latency values are relatively close, the spread of cold latency values is significantly larger. This highlights that the primary issue with cold starts is not just increased latency, but unpredictability.

This unpredictability becomes particularly important in real-world scenarios. In applications with intermittent or bursty traffic, instances frequently scale down to zero, increasing the likelihood of cold starts. As a result, users may experience occasional high-latency responses even if average performance appears acceptable.

From a system design perspective, this emphasizes the importance of instance reuse and strategies such as maintaining minimum instances or smoothing traffic patterns. Reducing the frequency of cold starts can significantly improve overall system reliability and user experience.

The key insight from this experiment is that cold starts introduce variability rather than just latency overhead. Therefore, optimizing serverless systems requires minimizing cold start frequency rather than focusing solely on reducing average response time.

5.2 Image Size vs Startup Latency

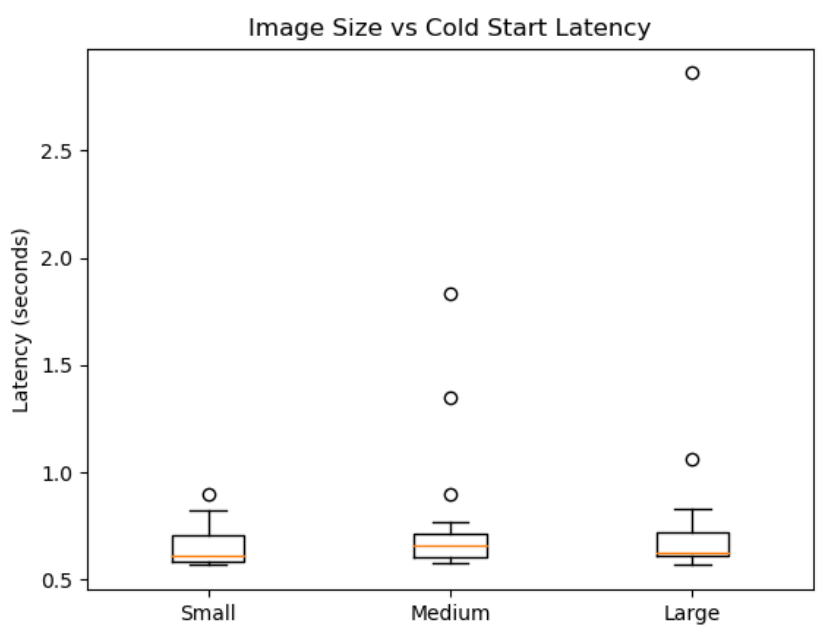
Container image size plays a significant role in determining startup latency in serverless environments. Larger images require more data to be transferred, unpacked, and initialized before the application can begin handling requests. This introduces additional overhead during cold starts.

Table 2: Image Size Impact

Image	Mean (seconds)	Observation
Small	0.65	Stable
Medium	0.73	Moderate Increase
Large	0.76	Plateau Observed

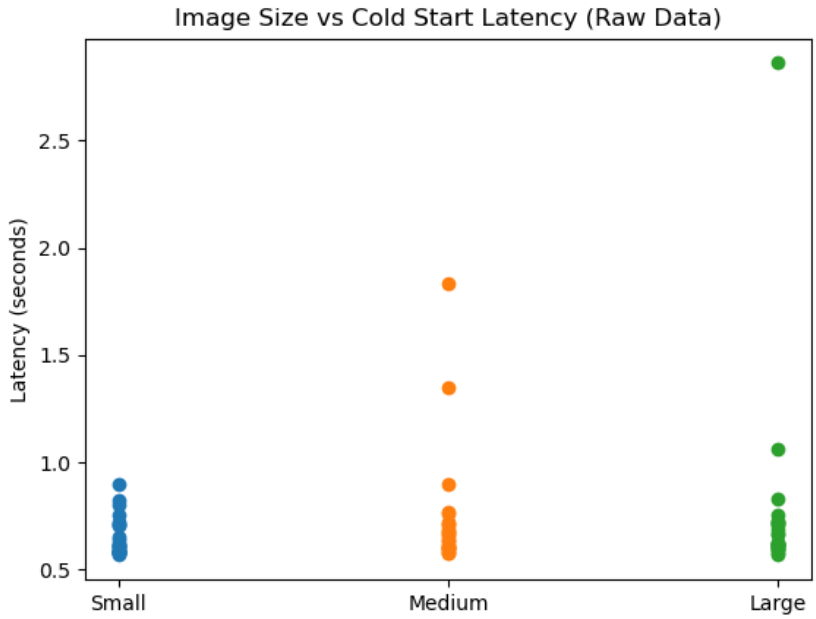
The table shows that startup latency increases as image size grows. However, the increase is not proportional. The difference between medium and large images is relatively small, suggesting diminishing returns in latency impact as size increases.

This behavior becomes clearer when analyzing the distribution of startup latency values.



The box plot indicates that while larger images tend to have slightly higher median latency, there is significant overlap between all categories. This suggests that image size is not the only factor influencing startup time.

A more detailed view is provided by the scatter plot.



The scatter distribution reveals that latency values for small, medium, and large images frequently overlap. Some medium images perform better than small ones, and some large images perform similarly to smaller ones.

This indicates the presence of underlying system optimizations, particularly caching mechanisms. Once a container image is pulled and stored within the platform's infrastructure, subsequent startups do not require full image transfer. Instead, cached layers are reused, significantly reducing startup overhead.

Additionally, factors such as network bandwidth, disk I/O performance, and container layer reuse contribute to this behavior. These factors introduce variability that can overshadow the impact of image size alone.

The key insight from this experiment is that while image size does influence startup latency, its effect is limited beyond initial deployments. Optimization efforts should therefore focus not only on reducing image size but also on improving caching efficiency and deployment frequency to ensure images remain warm within the system.

5.3 Concurrency Scaling

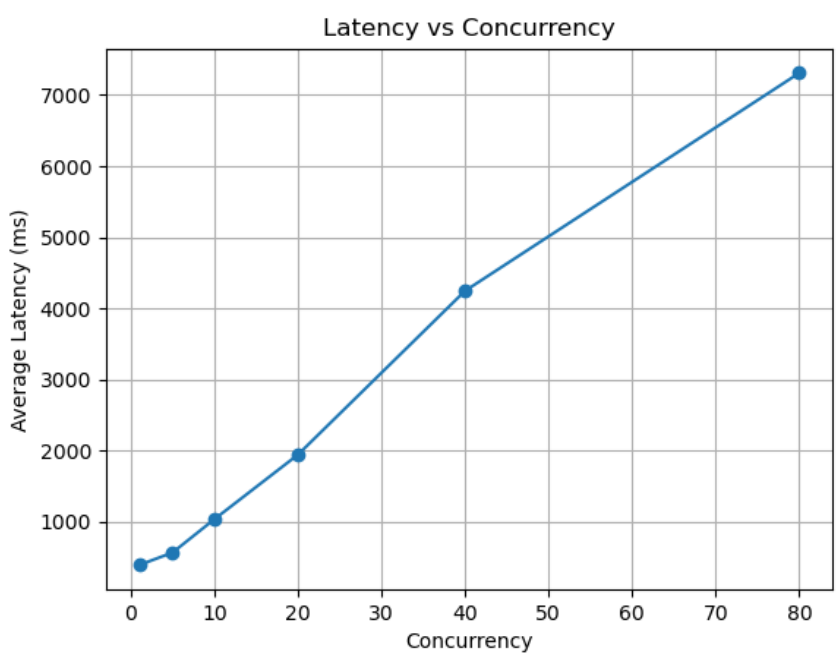
Concurrency is a critical factor in serverless systems as it directly determines how many requests must be handled simultaneously. Unlike traditional systems with fixed resources, serverless platforms dynamically scale by creating new instances in response to increasing load. However, this scaling process is not instantaneous, leading to observable performance variations under different concurrency levels.

Table 3: Concurrency Behavior

Metric	Value
Mean Latency	2578 ms
Max Latency	10992 ms
Threshold	∼40 concurrent requests

The table highlights a key observation: while average latency is already relatively high under concurrent load, the maximum latency is significantly larger, indicating the presence of severe outliers. The identified threshold of approximately 40 concurrent requests marks a transition point in system behavior.

This transition becomes more evident when analyzing latency as a function of concurrency.

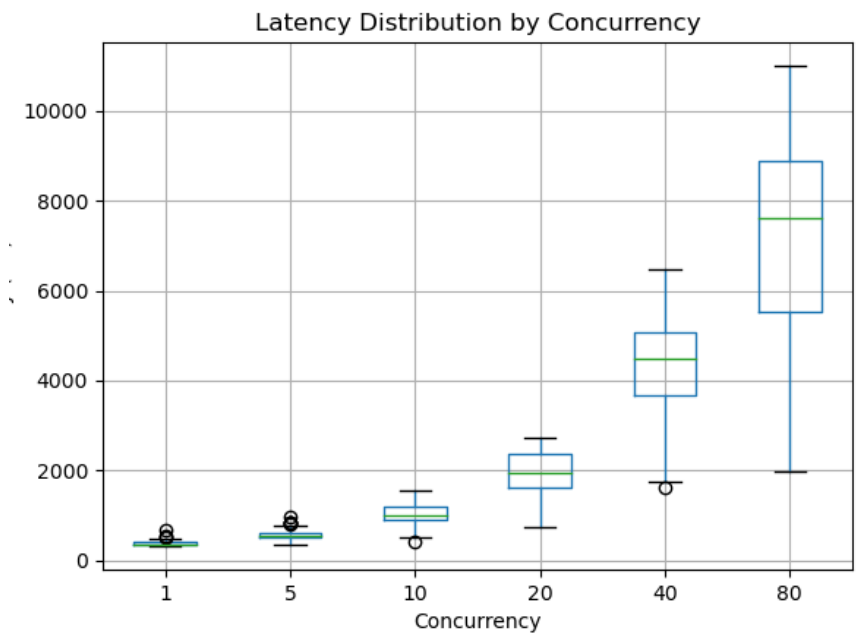


At lower concurrency levels, latency increases gradually. This is expected, as existing instances begin handling multiple requests, leading to slight queuing and increased processing

time. However, beyond the threshold, latency increases sharply, indicating that the system is no longer able to keep up with incoming demand.

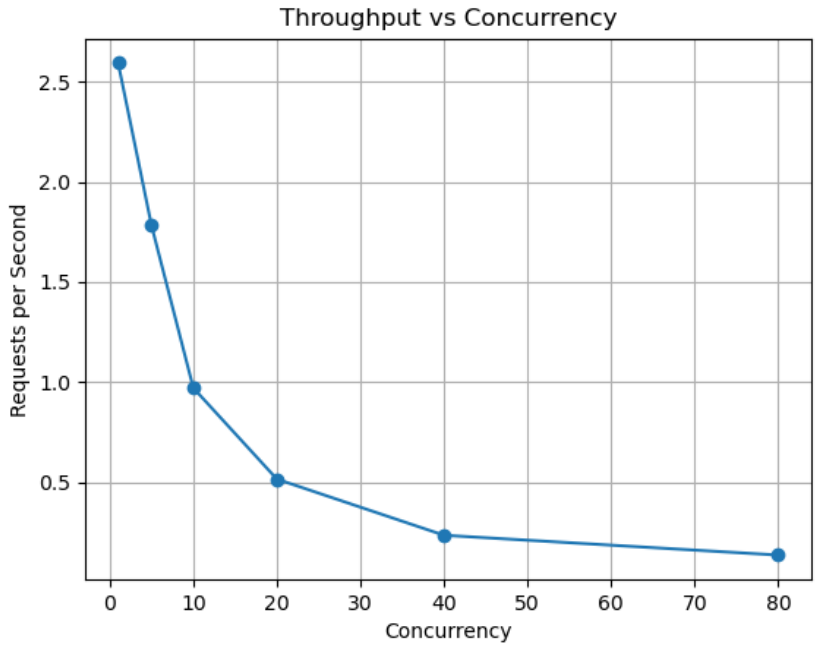
This sharp increase is primarily caused by delayed instance provisioning. When concurrency rises rapidly, the platform requires time to detect the increased load and spin up new instances. During this delay, incoming requests are queued, resulting in significantly higher response times.

The variability of latency under different concurrency levels is further illustrated in the distribution plot.



The box plot shows that as concurrency increases, the spread of latency values becomes wider. This indicates growing instability in system performance. At high concurrency levels, the presence of extreme outliers suggests that some requests experience substantial delays due to resource contention and queuing effects.

Throughput behavior provides additional insight into system capacity.



Initially, throughput increases as concurrency rises, reflecting improved resource utilization. However, after a certain point, throughput begins to plateau and eventually declines. This indicates that the system has reached its effective processing limit, where additional incoming requests no longer translate into higher output.

This phenomenon highlights a fundamental limitation of reactive scaling. Since the system scales in response to demand rather than anticipating it, there is always a lag between increased load and resource availability. During this lag, the system experiences temporary overload conditions, leading to increased latency and reduced efficiency.

From a system design perspective, this experiment reveals the existence of a concurrency threshold beyond which performance degrades rapidly. Operating near or beyond this threshold can result in unstable behavior, making it essential to understand and manage concurrency limits in serverless applications.

The key insight from this experiment is that serverless systems do not scale instantaneously. Instead, they exhibit threshold-based behavior, where performance remains stable up to a certain point and then degrades non-linearly. Designing systems that operate within this stable region is crucial for maintaining consistent performance.

5.4 Autoscaling Saturation

Autoscaling is a fundamental feature of serverless platforms, designed to dynamically adjust the number of running instances based on incoming traffic. In theory, this enables the system to handle arbitrary load without manual intervention. However, in practice, autoscaling operates in a reactive manner, introducing delays that significantly impact performance under high concurrency.

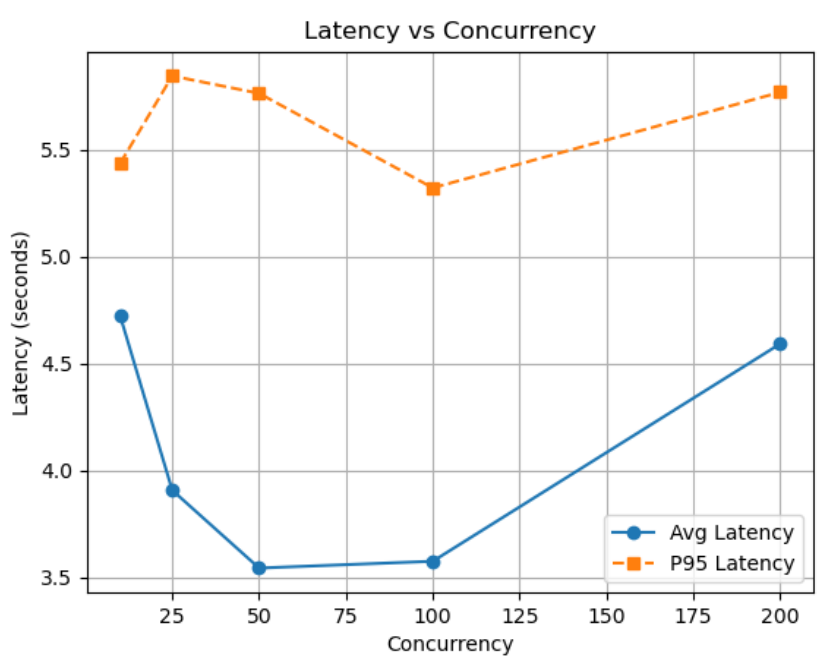
Table 4: Autoscaling Limits

Concurrency	Latency (seconds)	Error Rate	Throughput (req/s)
10	4.7	0.64	35
25	3.9	0.19	50
50	3.5	0.04	83

The table reveals an interesting trend in system behavior. At low concurrency levels, latency is relatively high and error rates are significant. This is primarily due to underutilization of resources, where available instances are not fully leveraged, leading to inefficient request handling.

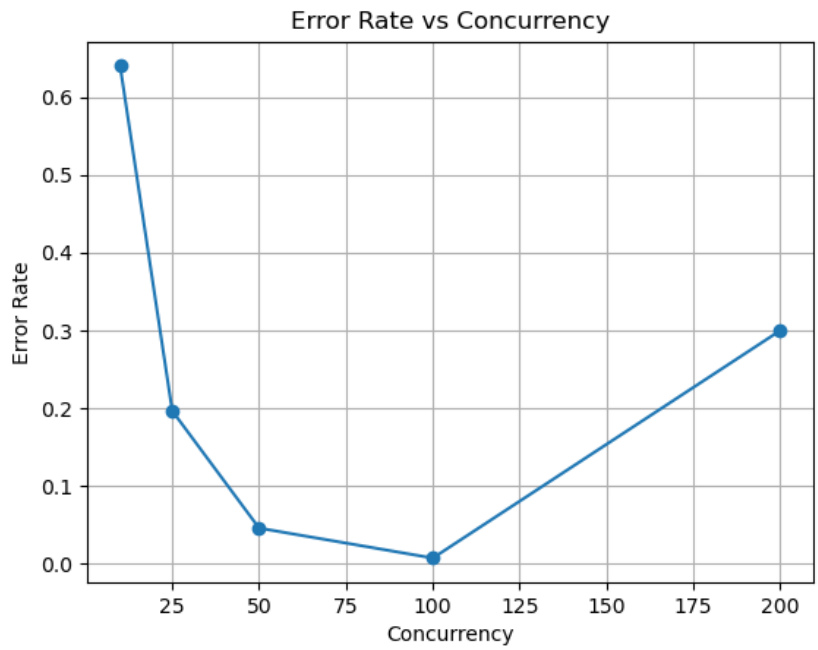
As concurrency increases to moderate levels, both latency and error rates decrease while throughput increases. This indicates that the system is entering an optimal operating region, where available resources are being utilized more effectively. Requests are distributed more efficiently across instances, reducing idle time and improving overall performance.

This trend is clearly visible in the latency behavior.



Initially, latency decreases as concurrency increases, reflecting improved efficiency. However, this trend does not continue indefinitely. As the system approaches higher concurrency levels, latency begins to rise again, indicating the onset of saturation.

The behavior of error rates provides deeper insight into this transition.



Error rates decrease significantly as the system moves from low to moderate load, confirming improved stability. However, beyond the optimal region, error rates begin to increase again. This indicates that the system is no longer able to handle incoming requests effectively, resulting in failures or dropped requests.

This phenomenon occurs because autoscaling is not instantaneous. When a sudden increase in requests occurs, the system requires time to detect the load change, allocate resources, and initialize new instances. During this delay, incoming requests accumulate, leading to increased latency and higher error rates.

Another important observation is the behavior of throughput. While throughput increases as concurrency rises, this increase is not indefinite. Eventually, the rate of throughput growth slows down, indicating that the system is reaching its processing limits.

This reflects the existence of a saturation point, where additional load does not result in proportional increases in system capacity. Instead, the system becomes constrained by factors such as instance provisioning speed, CPU limits, and request queuing mechanisms.

From a system design perspective, these results highlight the presence of three distinct operating regions:

- **Underutilized Region:** Low concurrency, high latency, inefficient resource usage.
- **Optimal Region:** Moderate concurrency, low latency, high throughput.
- **Saturation Region:** High concurrency, increasing latency and error rates.

Understanding these regions is critical for designing reliable serverless systems. Operating within the optimal region ensures efficient performance, while exceeding this region can lead to rapid degradation.

The key insight from this experiment is that autoscaling does not eliminate performance limits; it only shifts them. Even with automatic scaling, there exists a practical upper bound beyond which system performance deteriorates due to delayed resource provisioning and inherent infrastructure constraints.

5.5 Regional Latency

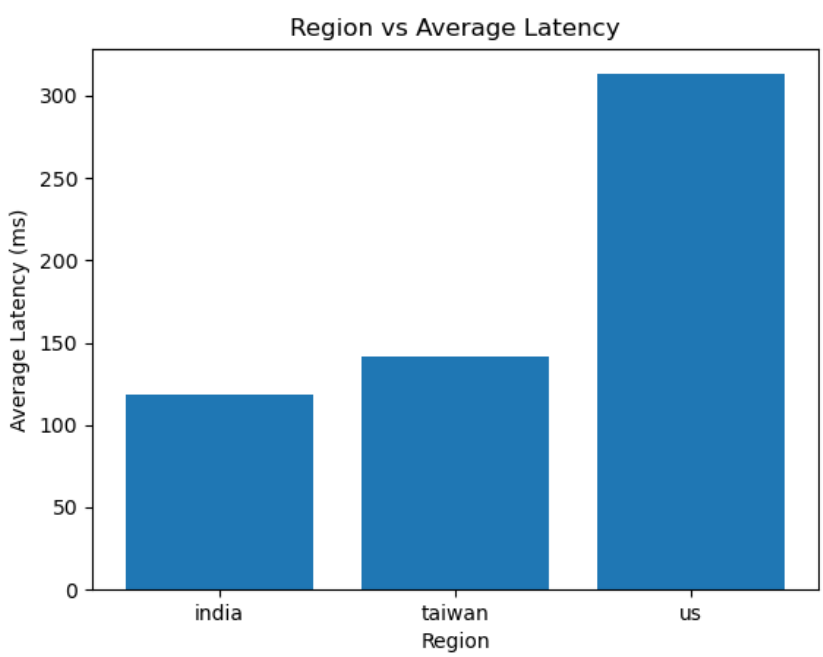
Regional deployment plays a critical role in serverless performance due to the distributed nature of cloud infrastructure. Unlike compute-related factors such as scaling and concurrency, regional latency is primarily influenced by network characteristics, including physical distance, routing paths, and network congestion.

Table 5: Regional Performance

Region	Avg (ms)	Max (ms)	Throughput (req/s)
India	118	1072	41
Taiwan	141	676	34
US	313	830	15

The table clearly shows that average latency increases as the deployment region moves farther from the client. The India region exhibits the lowest average latency, followed by Taiwan, while the US region shows significantly higher latency. This trend aligns with expectations based on geographic distance and network propagation delay.

However, average latency alone does not fully capture system behavior. The maximum latency values reveal an important insight: even geographically closer regions experience significant spikes.

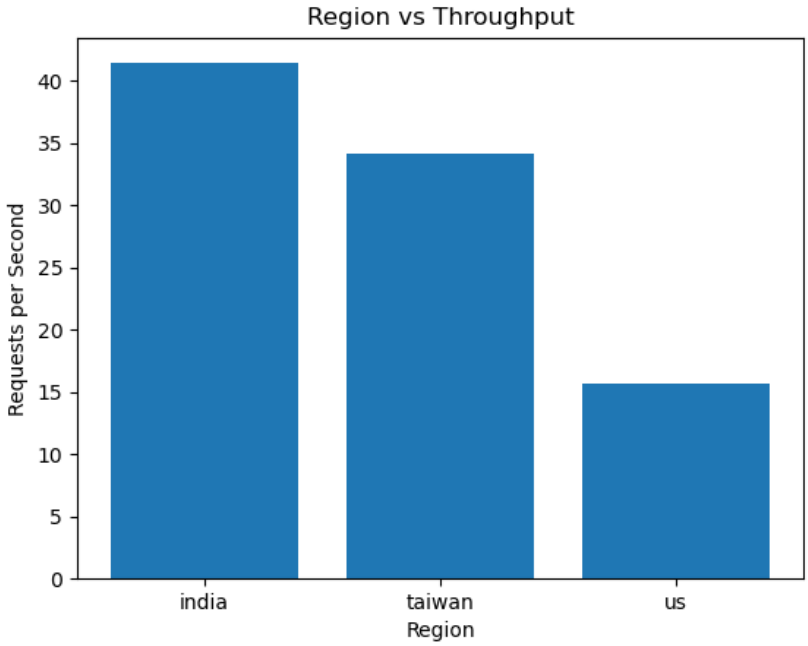


The latency graph highlights that while closer regions generally perform better, variability exists across all regions. These fluctuations are caused by dynamic network conditions,

including congestion, packet routing changes, and intermediate node delays.

Unlike compute-related latency, which is influenced by system scaling and resource allocation, network latency is affected by factors outside the control of the serverless platform. As a result, even optimally deployed services may experience occasional spikes.

Throughput differences further illustrate the impact of regional factors.



Regions with lower latency generally achieve higher throughput, as requests are completed more quickly and system resources are freed sooner. In contrast, higher latency regions experience reduced throughput due to longer request lifetimes and increased waiting times.

An important observation is that geographic proximity improves average performance but does not eliminate variability. Even the closest region exhibits occasional high-latency responses, indicating that network behavior is inherently unpredictable.

From a system design perspective, this suggests that deploying services closer to users is necessary but not sufficient for achieving consistent performance. Additional strategies such as multi-region deployment, load balancing, and caching may be required to mitigate network-induced variability.

The key insight from this experiment is that regional latency is governed by both distance and dynamic network behavior. While proximity reduces baseline latency, variability remains an inherent characteristic of distributed systems and must be accounted for in application design.

6 Discussion

The results of this study highlight that serverless performance is governed by a combination of dynamic system behavior and external factors rather than static metrics such as average latency. Across all experiments, a recurring theme is the presence of variability and threshold-based behavior, which significantly influence real-world performance.

Cold start analysis demonstrates that unpredictability is a more critical concern than average latency. While mean response times remain within acceptable limits, sporadic high-latency events can degrade user experience, particularly in applications requiring consistent response times. This reinforces the importance of instance reuse and workload stabilization strategies.

The experiments on concurrency and autoscaling reveal that serverless systems exhibit non-linear performance characteristics. Instead of scaling smoothly, the system operates within distinct regions: an underutilized region with inefficient resource usage, an optimal region with stable performance, and a saturation region where performance degrades rapidly. This behavior is primarily driven by the reactive nature of autoscaling, where resource provisioning lags behind demand.

Furthermore, the analysis of image size indicates that optimization efforts should not focus solely on reducing container size. While image size affects initial startup latency, caching mechanisms and infrastructure-level optimizations significantly reduce its impact over time.

Regional latency experiments introduce an additional dimension by highlighting the role of network variability. Even when services are deployed close to users, fluctuations in network conditions can introduce latency spikes. This demonstrates that performance in distributed systems is influenced by factors beyond compute resources.

Overall, the findings suggest that designing efficient serverless systems requires a holistic approach. Developers must consider not only application logic but also workload patterns, deployment strategies, and infrastructure behavior to achieve consistent performance.

7 Limitations

While this study provides a comprehensive empirical analysis of serverless performance, it is subject to several limitations.

First, all experiments were conducted on a single platform, Google Cloud Run. Different serverless platforms such as AWS Lambda or Azure Functions may exhibit different scaling behaviors, resource allocation strategies, and performance characteristics.

Second, the experiments were performed under controlled workload conditions. Real-world traffic patterns are often more complex, including sudden bursts, long idle periods, and unpredictable request distributions. These factors may influence system behavior in ways not fully captured in this study.

Third, the analysis focuses primarily on latency, throughput, and error rates. Other important factors such as cost efficiency, memory utilization, and cold start mitigation techniques were not explored in detail.

Additionally, network conditions during the experiments may have introduced variability, particularly in regional latency measurements. While this reflects real-world behavior, it also introduces uncertainty that is difficult to fully control.

Finally, the experiments were conducted using a specific application setup and container configuration. Different application architectures, programming languages, and dependency sizes may lead to different performance outcomes.

These limitations indicate that while the findings provide valuable insights, they should be interpreted within the context of the experimental setup.

8 Conclusion

This paper presents a detailed empirical study of serverless performance, focusing on key factors such as cold start behavior, container image size, concurrency scaling, autoscaling saturation, and regional latency.

The results demonstrate that serverless systems exhibit complex, non-linear behavior rather than simple, predictable scaling. While average latency remains moderate, variability and tail latency play a significant role in determining overall system performance. Cold starts introduce unpredictability, concurrency reveals threshold-based degradation, and autoscaling exposes inherent limits in resource provisioning.

The study also highlights that performance optimization in serverless systems requires more than isolated improvements. Reducing image size, for example, has limited impact compared to understanding caching mechanisms and workload patterns. Similarly, deploying services closer to users improves average latency but does not eliminate network-induced variability.

A key takeaway from this work is that serverless platforms do not provide infinite scalability. Instead, they operate within practical limits defined by infrastructure constraints and scaling delays. Designing robust serverless applications therefore requires careful consideration of these limits, along with strategies to mitigate variability and maintain stability.

Future work can extend this study by exploring multi-platform comparisons, cost-performance trade-offs, and advanced optimization techniques such as predictive scaling and workload shaping.

In conclusion, understanding the dynamic behavior of serverless systems is essential for building reliable and efficient applications. This study provides a foundation for such understanding by combining empirical measurements with detailed analysis of underlying system behavior.